Popàp-i

DISIM - Università degli Studi dell'Aquila

Street Science 2025

Premessa

Questo problema si ispira a quelli della categoria "Robot e Automi" che potete incontrare nelle gare delle Olimpiadi del Problem Solving (OPS), sebbene con alcune licenze. Trovate tutte le istruzioni sulla Guida OPS e nella sezione dedicata.

Problema

Più tardi andrete a vedere il nostro pop-up al Castello (vero?). Quello che non sapete è che i preparativi stanno fervendo proprio in questo momento, e non ci dispiacerebbe un po' di aiuto col testing.

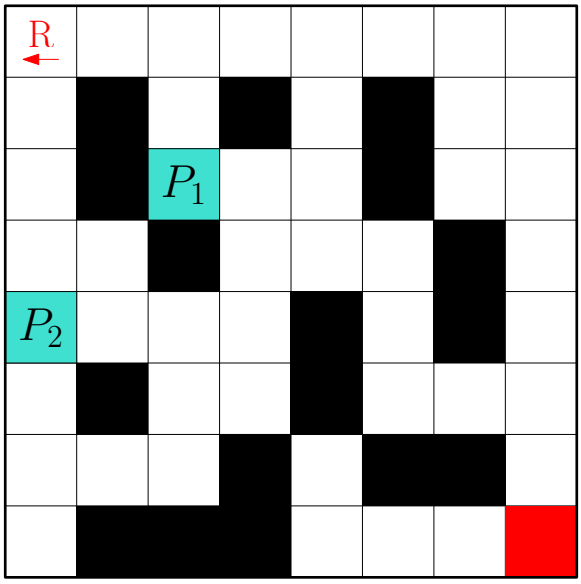
Si tratta di capire se le attività progettate per i visitatori funzionino come ci aspettiamo. In particolare, vogliamo assicurarci che vada tutto bene coi nostri robot. Ormai abbiamo la testa fusa, e non riusciamo ad accordarci sull'esito dell'esperimento proposto allo stand: ognuno di noi ne suggerisce uno diverso!

Vi diamo tre mappe e, per ognuna, riportiamo le nostre incertezze. Aiutateci voi.

Domande Gigi ha progettato un robottino in grado di evitare ostacoli e raccogliere monete dal campo di gara. Nella griglia qui sotto, i primi corrispondono alle caselle nere, e le monete sono indicate da 1. Il robottino parte in alto a sinistra e deve arrivare sulla casella rossa, ma può muoversi solo verso destra e verso il basso! Dateci la **lista di comandi** del robot che gli permetta di raccogliere più monete possibili nel suo tragitto. Potete usare i classici comandi **f**, **a** e **o**. Se avete due o più comandi f di fila, **uniteli in un comando r**! (**0.3 punti**)

R →	1	0	0	1	0	0	1
0	0	0	0		0	1	1
1		0	0	0	0	1	1
0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1
0		1	0	0	0	1	0
0	0		0	1	0	0	1
0	0	0	0	0	0	1	

Fulvio ha scritto del codice che fa sì che il robot arrivi a destinazione toccando prima delle **tappe obbligatorie**. Nel diagramma in basso, queste corrispondono alle caselle P_1 e P_2. Ci piacerebbe mostrare ai visitatori anche questa funzionalità, quindi vogliamo verificare che il suo algoritmo sia corretto. Vi chiediamo una **lista di comandi** *di lunghezza minima* che porti il robot dalla casella in alto a sinistra a quella rossa garantendo che le due tappe obbligatorie siano toccate, in qualsiasi ordine. Potete usare i classici comandi **f**, **a** e **o**. Se avete due o più comandi f di fila, **uniteli in un comando r**! (**0.4 punti**)



Il terzo e ultimo compito del robottino consiste nel partire da una casella **e tornare sulla stessa**, dopo aver raggiunto delle **tappe intermedie obbligatorie** e rispettando delle **regole di movimento**.
La mappa di riferimento è quella in basso (**notate** che l'asse y è ribaltato, quindi le ordinate "vanno verso il basso"). Vogliamo che il robot parta dalla casella rossa in alto a sinistra, tocchi tutte le caselle con una S blu in qualsiasi ordine e poi vi faccia ritorno. **Attenzione!** Il robot non si muove nel modo consueto. Ogni casella ospita un numeretto: se il robot *si ferma* su una casella che ospita il numero k, il suo successivo spostamento consiste in un **salto** di k caselle in una delle quattro direzioni possibili. Per esempio, dalla casella $(3, 2)$ può *saltare* in $(1, 2)$ o $(5, 2)$. Vi chiediamo di darci la **lista delle posizioni** progressivamente occupate dal robot in un percorso valido che sia composto da *meno di 25 salti*. (**0.3 punti**)

	1	2	3	4	5	6	7	8	9	10
1	**R:2**	1	1	1	3	1	2		1	1
2	1		2	1	1	2	1	2		3
3	1	1	3	1	2		3	1	1	2
4		2		2	1	1		2	1	1
5	1	1	1		**S:2**	1	2		2	3
6	2		3	1	2	1		3	1	2
7	1	2	1	1		2	1	2	1	**S:3**
8	2	1		2	1	3	1		2	1
9	3	1	2		1	1	2	1	3	1
10	1		1	2	1	**S:2**	2	1		2

Formato delle Risposte

La risposta ai primi due punti consiste in una lista di comandi nel consueto formato [comando$_1$, ..., comando$_n$]. Ogni lista deve essere data sotto forma di una *stringa*, su una sola riga. Le risposte devono essere ognuna su una riga dedicata. Se volete omettere una risposta, lasciate una riga vuota.

La risposta al terzo punto in una lista di posizioni nel formato [pos$_1$, ..., pos$_n$], dove ogni posizione è della forma familiare [X,Y]. Di nuovo, ogni lista deve essere una *stringa*, su una sola riga. Se volete omettere la risposta, lasciate una riga vuota.

Importante Nelle risposte che inserite sulla piattaforma di correzione non è ammesso nessun carattere non esplicitamente richiesto. Fate attenzione a caratteri indesiderati prima, dopo e dentro la risposta.

Non è garantito che risposte corrette ma non conformi a quanto specificato vengano considerate giuste!

Soluzione

La risposta al primo quesito è la lista di comandi

$$L_1 = [\text{r7f|,o,r7f|}]$$

La risposta al secondo quesito è, invece, la lista di comandi

$$L_2 = [\text{a,r4f|,a,r3f|,a,r2f|,a,f,a,a,r2f|,o,f,a,f,o,r2f|,a,r2f|,o,r2f|}]$$

Infine, una lista di posizioni come richiesta al terzo punto è:

L_3 = [(1,1),(3,1),(4,1),(5,1),(5,4),**(5,5)**,(7,5),(9,5),(9,7),**(10,7)**,(7,7),(6,7),(6,9),**(6,10)**,(6,8),(6,5),(5,5),(5,3),(5,1),(2,1),(1,1)]

Commento alla Soluzione

Per rispondere al primo quesito, ragioniamo su come è fatta la mappa. Osserviamo che l'1 nella prima colonna non è conveniente. Infatti, la mossa successiva è obbligatoriamente una mossa verso il basso, dopo la quale non c'è alcun modo di guadagnare più di quattro monete aggiuntive. Ma nella griglia c'è una colonna che, da sola, contiene più di $4 + 1 = 5$ monete! Si tratta della colonna più a destra. È facile osservare che la colonna si può raggiungere semplicemente andando sempre a destra. Quindi la (una) lista cercata è semplicemente L_1 = [r7f|,o,r7f|].

Passiamo al secondo quesito. Facciamo un'osservazione fondamentale. Il percorso cercato deve essere della forma:

- percorso più veloce dalla partenza a P_1 (P_2);
- percorso più veloce da P_1 (P_2) a P_2 (P_1);
- percorso più veloce da P_2 alla casella rossa.

Analizziamo quindi le due possibilità alternative: andare prima in P_1 o andare prima in P_2.

Prima P_1 È facile osservare che il modo più veloce per andare consta di 7 mosse. A questo punto, la via più rapida per P_2 è quella "verso il basso"", di 9 mosse. Ora, è possibile raggiungere il cerchio rosso con non meno di 19 mosse. Questa alternativa non può richiedere meno di 35 mosse.
Prima P_2 Per raggiungere P_2 basta ruotare e scendere, in sole 5 mosse. Ora sono necessarie altre 9 mosse per andare in P_1. Infine, ne bastano 17 per arrivare alla casella rossa. Questa seconda alternativa può richiedere sole 31 mosse. Scegliamo, quindi, questa opzione, e costruiamo la lista corrispondente usando i comandi **r** per evitare comandi **f** consecutivi: L_2 = [a,r4f|,a,r3f|,a,r2f|,a,f,a,a,r2f|,o,f,a,f,o,r2f|,a,r2f|,o,r2f|].

Per questo punto non diamo ulteriori informazioni. Trovate altre liste, se ce ne sono.